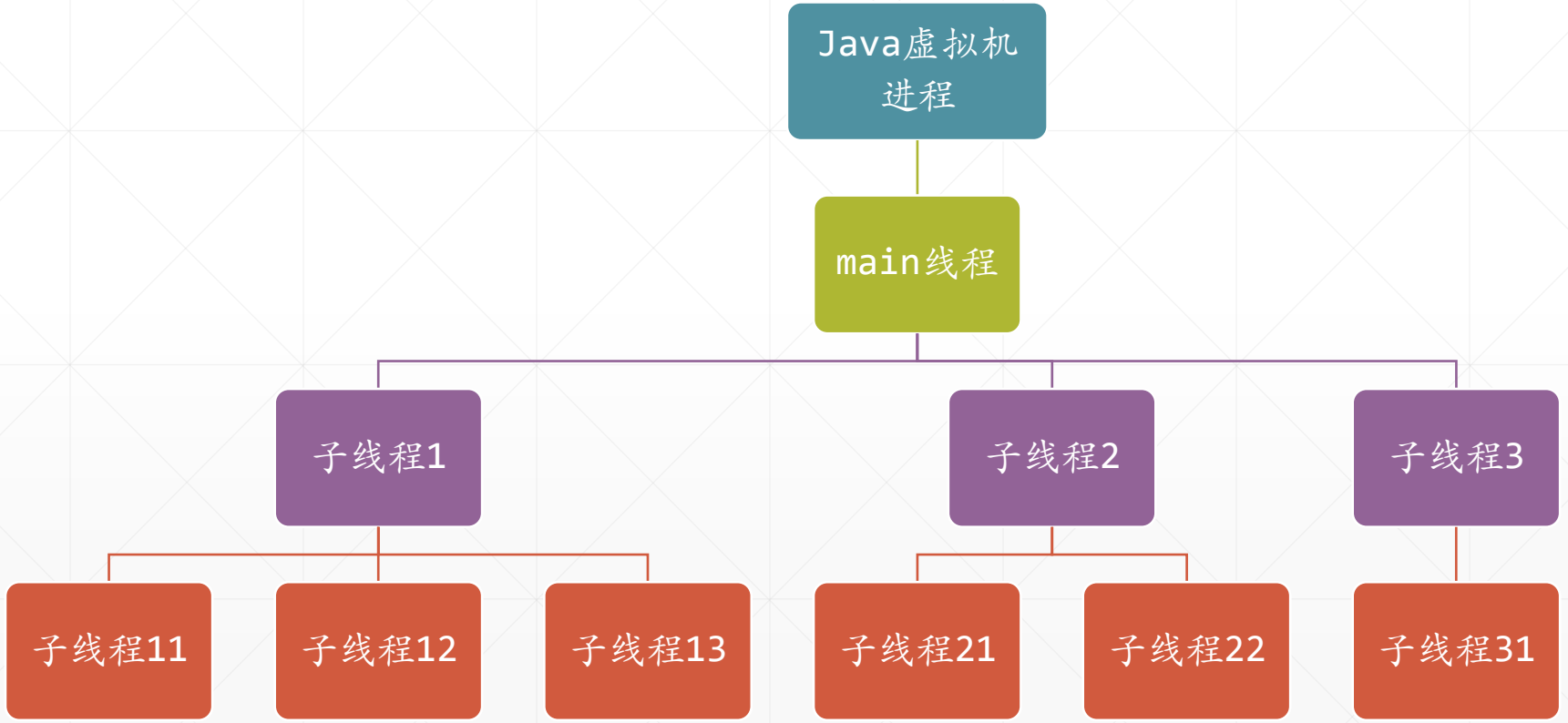
父子线程与线程组

北京理工大学计算机学院
金旭亮

线程的父子关系

线程A创建线程B，则A是B的“父线程”，B是A的“子线程”。



Thread.java

```
379     /** Initializes a Thread. ...*/
        3 usages
392     @SuppressWarnings("removal")
393 @   private Thread(ThreadGroup g, Runnable target, String name,
394                   long stackSize, AccessControlContext acc,
395                   boolean inheritThreadLocals) {
396         if (name == null) {
397             throw new NullPointerException("name cannot be null");
398         }
399
400         this.name = name;
401
402         Thread parent = currentThread();
403         SecurityManager security = System.getSecurityManager();
404         if (g == null) {...}
```

在Thread构造函数中，设置“新”线程的parent为“老”线程

父线程对子线程的影响

- 守护线程创建的子线程，如果不专门指定，其子线程也是守护线程。
- 一个线程的优先级，默认值为父线程的优先级。

父线程对子线程只影响其特定的属性值，一旦子线程创建完毕并开始运行，父子线程之间就再无关联。

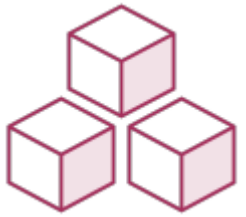
主线程与后台线程

- 在Java应用程序中，Java虚拟机创建的第一个线程，通常称为“**主线程（main thread）**”。

- 在有可视化界面的桌面应用程序中，窗体就是由**主线程（main thread）**创建的，为了能及时响应用户的键盘和鼠标等操作，主线程通常会创建一些子线程，负责完成一些耗时的任务，这些子线程，通常称为“**后台线程（background thread）**”或“**工作者线程（worker thread）**”。

线程组

线程组(Thread Groups)



Java中的任何一个线程，都有一个线程组与之关联，这个线程组对象的引用，可以通过Thread.getThreadGroup()调用来获取。

如果创建线程时没有指定线程组，那么这个线程就属于它的父线程所属的线程组。

ThreadGroup的主要职责：

- 将多个相关线程归作一组
- 每个线程组都有一个唯一的名字

线程组并不能直接干涉线程的运行，它主要功能是对线程进行分组以方便管理。

查看相关的源码

Thread.java

```
784     public synchronized void start() {
785         /** This method is not invoked for the main method thread or "system" ...*/
792         if (threadStatus != 0)
793             throw new IllegalThreadStateException();
794
795         /* Notify the group that this thread is about to be started
796          * so that it can be added to the group's list of threads
797          * and the group's unstarted count can be decremented. */
798         group.add(this);
799
800         boolean started = false;
801         try {...} finally {
805             try {...} catch (Throwable ignore) {
810                 /*...*/
812             }
813         }
814     }
```

在Thread.start()方法源码中可以清楚地看到，每个线程都有一个线程组。

线程组的创建

（1）定义（父子）线程组：

```
ThreadGroup parent = new ThreadGroup("parent");
ThreadGroup child = new ThreadGroup(parent, "Child");
```

（2）向线程组中增加新线程：

```
Thread t1 = new Thread (parent, "t1");
Thread t2 = new Thread (child, "t2");
```

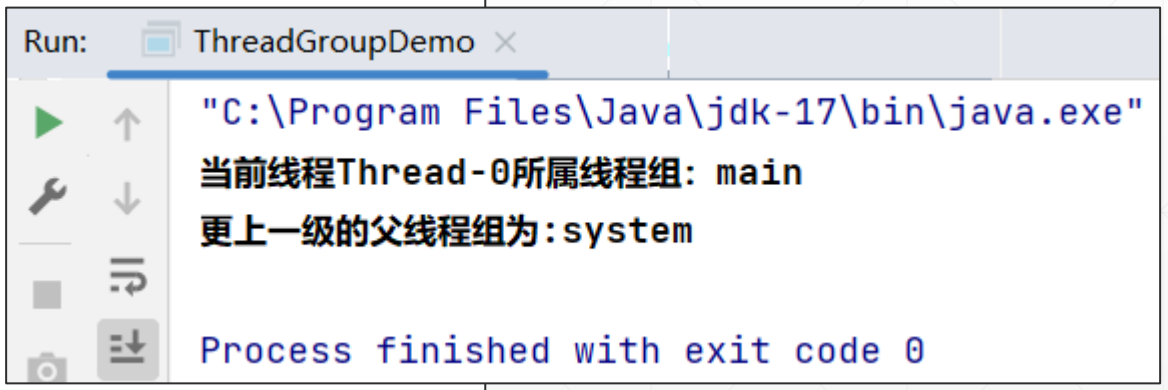
线程与线程组之间的关系，类似于文件与文件夹之间的关系。

了解线程组的父子关系

ThreadGroupDemo.java

```
private static void parentThreadGroup() {
    Runnable runnable = () -> {
        var threadName : String = Thread.currentThread().getName();
        var currentGroup : ThreadGroup = Thread.currentThread().getThreadGroup();
        System.out.println("当前线程" + threadName + "所属线程组: "
                + currentGroup.getName());
        for (; ; ) {
            var parentGroup : ThreadGroup = currentGroup.getParent();
            if (parentGroup != null) {
                System.out.println("更上一级的父线程组为:" + parentGroup.getName());
                currentGroup = parentGroup;
            } else {
                break;
            }
        }
    };
    new Thread(runnable).start();
}
```

在main方法中调用左侧函数，可以得到以下的输出：



查询线程与线程组的隶属关系

```
1 usage
private static void threadGroupInfo() {
    Runnable runnable = () -> {
        String groupName = Thread.currentThread().getThreadGroup().getName();
        String threadName = Thread.currentThread().getName();
        System.out.println("\n" + threadName + "属于" + groupName);
    };
    ThreadGroup group = new ThreadGroup( name: "MyThreadGoup");
    Thread thread1 = new Thread(group, runnable);
    Thread thread2 = new Thread(group, runnable);
    thread1.start();
    thread2.start();
    System.out.println("线程组有激活的线程: " + group.activeCount());
    System.out.println("-----------");
    group.list();
    System.out.println("-----------");
}
```

ThreadGroupDemo.java

```
Run:  ThreadGroupDemo
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent
线程组有激活的线程: 2
-----------
java.lang.ThreadGroup[name=MyThreadGoup,maxpri=10]
    Thread[Thread-0,5,MyThreadGoup]
    Thread[Thread-1,5,MyThreadGoup]
-----------

Thread-0属于MyThreadGoup

Thread-1属于MyThreadGoup

Process finished with exit code 0
```

示例：中断线程组中的所有线程-1

“线程组”具备“消息转发”功能。

```
//线程将要执行的函数
1 usage
private static void task() {
    var threadName = "线程" + Thread.currentThread().getName();
    System.out.println(threadName + "开始运行...");
    while (true) {
        try {
            TimeUnit.MILLISECONDS.sleep( timeout: 100);
        } catch (InterruptedException e) {
            break;
        }
    }
    System.out.println(threadName + "已退出");
}
```

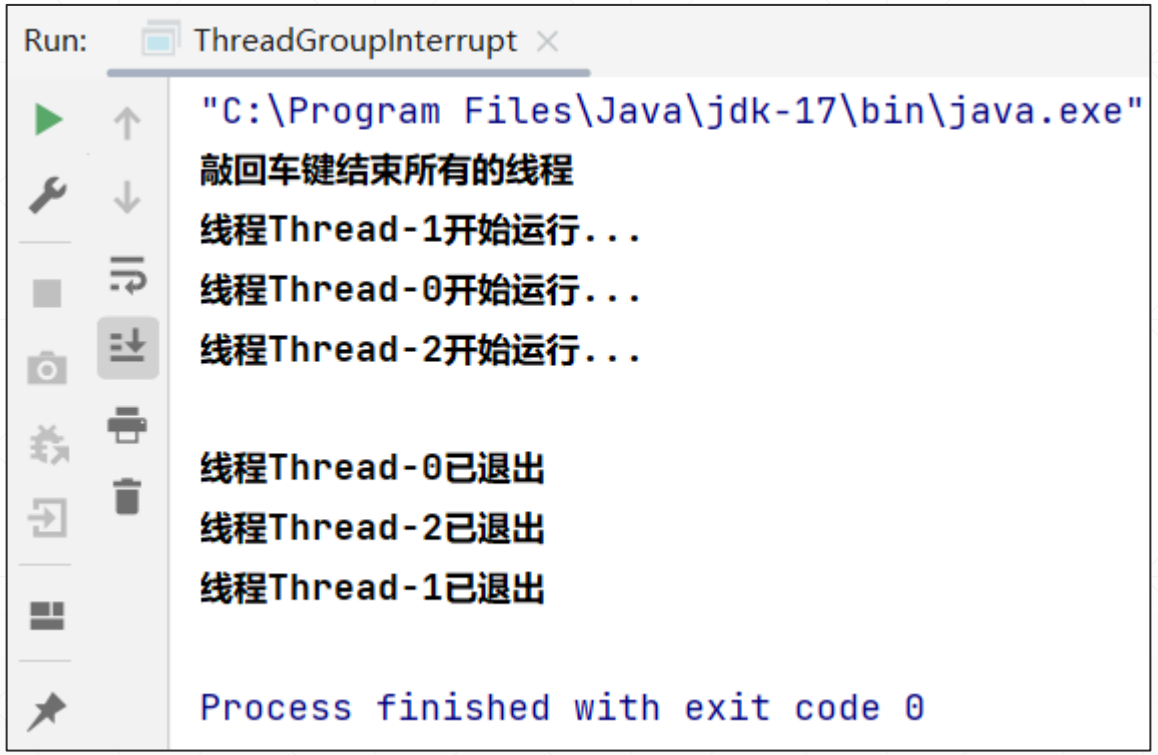
sleep方法能响应外部的“中断（interrupt）”请求

ThreadGroupInterrupt.java

示例：中断线程组中的所有线程-2

```java
public class ThreadGroupInterrupt {
    public static void main(String[] args) {
        //创建一个线程组
        var group = new ThreadGroup( name: "TestGroup");
        //启动三个线程，并将其置于同一个线程组中
        Runnable runnable = ThreadGroupInterrupt::task;
        for (int i = 0; i < 3; i++) {
            new Thread(group, runnable).start();
        }
        System.out.println("敲回车键结束所有的线程");
        new Scanner(System.in).nextLine();
        //向线程组发出中断请求
        group.interrupt();
    }

    //线程将要执行的函数
    1 usage
    private static void task() {...}
}
```



可以看到，向线程组发出的中断请求，会被“自动转发”给组中所有的线程。

使用线程组捕获线程异常

ThreadGroupCatchException.java

```java
public class ThreadGroupCatchException {
    public static void main(String[] args) {
        ThreadGroup threadGroup = new ThreadGroup( name: "myGroup") {
                    // 继承ThreadGroup并重新定义uncaughtException方法
                    // 在线程成员抛出unchecked exception时会执行此方法
                    public void uncaughtException(Thread t, Throwable e) {
                        System.out.println(t.getName() + ": " + e.getMessage());
                    }
                };
        Thread thread = new Thread(threadGroup, () -> {
            // 抛出unchecked异常
            throw new RuntimeException("用于测试异常捕获而有意抛出的异常");
        });
        thread.start();
    }
}
```

Run: ThreadGroupCatchException

```
"C:\Program Files\Java\jdk-17\bin\java.exe"
Thread-0: 用于测试异常捕获而有意抛出的异常

Process finished with exit code 0
```